

Deben recordar que los proyectos deben ser planteados bajo el contexto de I+D+i (Investigación + Desarrollo + innovación)

El proyecto debe definirse a partir de:

1. Clientes reales solucionando una problemática actual, la solución debe ser a través de las TIC.
2. Algún sector de la industria donde ustedes vean una oportunidad de negocio que pueda ser cubierta por las TIC.
3. Una necesidad no cubierta que puedan resolver utilizando las TIC.
4. Al plantear el problema y el negocio pueden considerar:

Puntos a agregar en su análisis	¿Por qué conviene incluirlo?
Declaración precisa del problema (causas, efectos, tamaño del impacto)	Sirve para medir después si la solución realmente resuelve algo significativo.
Objetivos SMART (específicos, medibles...)	Facilitan la evaluación y trazabilidad de logros.
Alcance y exclusiones	Evita que el proyecto se expanda sin control («scope creep»).
Análisis de partes interesadas externas (stakeholders): personas, grupos u organizaciones ajenas al equipo de desarrollo que influyen, aportan o se ven afectadas por el proyecto. Pueden incluir usuarios finales, clientes, expertos del dominio, patrocinadores, reguladores, etc.	Alinea expectativas y define responsabilidades tempranamente.
Evaluación de viabilidad y de modelo de negocio (canvas, análisis costo-beneficio, proyección de ROI)	Aporta la mirada de sustentabilidad comercial que esperas en los puntos 1–3.
Revisión de literatura / estado del arte académica y de mercado	Demuestra rigor y justifica la novedad; nutre el benchmarking.

Con respecto a la solución propuesta:

1. Si existen soluciones equivalentes, deben agregar su propuesta de valor ¿qué diferencia su solución de las otras? (benchmarking).
2. Con respecto a los requisitos y diseño de la solución:

Puntos a considerar	¿Por qué conviene incluirlo?
Especificación de requisitos (funcionales, no funcionales, priorizados con MoSCoW o Kano)	Aclara expectativas y define criterios de aceptación.
Arquitectura de software (diagramas C4, vistas 4+1, decisión de microservicios vs monolito, patrones)	Sustenta la robustez, escalabilidad y mantenibilidad.
Modelo de dominio + diccionario de datos	Complementa tu “modelo de datos robusto” y evita ambigüedades conceptuales.
Política de seguridad (autenticación, autorización, cifrado, OWASP Top 10)	Fundamental cuando manejas datos reales y APIs públicas.
Gobernanza de datos & cumplimiento legal (LGPD/GDPR, Ley 19.628 en Chile, licencias de datos)	Protege la tesis y la futura explotación comercial.
Accesibilidad (WCAG 2.2) y usabilidad	Eleva la calidad y la inclusividad; frecuentemente es un criterio de evaluación externa.
Diseño de experiencia de usuario (UX) con prototipos de baja/alta fidelidad	Permite tests tempranos con stakeholders.
Plan de integración y despliegue (Docker, Kubernetes, serverless, on-prem)	Aclara la viabilidad técnica y facilita demos.

Desarrollo y aseguramiento de calidad

1. Consideren que es un proyecto de Ingeniería, la programación solo es una componente que permite obtener un producto final.
2. La documentación del proceso es fundamental, así como el compromiso de los stakeholders en este proceso.
3. Les recomiendo considerar puntos como:

Puntos a considerar en el desarrollo y aseguramiento de calidad	¿Por qué conviene incluirlo?
Metodología de desarrollo (Scrum, Kanban, DevOps) con bitácora de sprints	Demuestra gestión de proyecto de ingeniería, no sólo codificación.
Repositorio con control de versiones (Git) y flujo de trabajo (GitFlow, trunk-based)	Evidencia colaboración y buenas prácticas.
CI/CD automático (GitHub Actions, GitLab CI, Jenkins)	Refuerza la cultura DevOps y la reutilización.
Cobertura de pruebas: unitarias, integración, E2E, pruebas de rendimiento	Alinea con la robustez y las reglas de negocio exigidas.
Monitoreo y logging (ELK, Prometheus, OpenTelemetry)	Permite evaluar la solución en ambiente real.
Métricas de calidad de código (SonarQube, linters)	Facilitan retroalimentación objetiva durante la tesis.

Validación y evaluación de resultados

La fase de validación y evaluación de resultados debe incluir	¿Por qué conviene incluirlo?
Pruebas con usuarios (test de usabilidad, SUS, Net Promoter Score)	Aporta evidencia de aceptación y valor percibido.
KPIs de negocio definidos y medidos (ej.: tiempo de proceso, ingresos)	Conectan la solución con la problemática original.
Comparación cuantitativa con soluciones existentes (tiempos, precisión, costos)	Refuerza el benchmarking y la propuesta de valor.
Análisis de riesgos y plan de mitigación	Muestra visión integral y profesional.
Impacto ético y social (algoritmos justos, sesgos, sostenibilidad)	Tema cada vez más evaluado en acreditaciones y conferencias.

Entregables y documentación

Entregable clave	Contenido mínimo
Plan de proyecto	Cronograma, presupuesto, RACI, matriz de riesgos.
Documentos técnicos	Especificación de requisitos, modelo de datos, diseño arquitectónico, API spec (OpenAPI), manual de instalación, manual de usuario, plan de pruebas, resultados de pruebas.
Demo funcional + video de la solución	Demostración para comisión y futuros clientes.
Repositorio público/privado	Código fuente, pipelines, issues.
Paper o artículo corto (formato IEEE/ACM)	Útil para difusión académica y puntaje curricular (CV).

La solución debe incorporar como mínimo, desde el punto de vista de las tecnologías:

1. Modelo de datos robusto.
2. Integración de plataformas web y mobile.(Back Office y Front Office) a través de api's propias.
3. Conexión a base de datos propia.
4. CRUD de todas las tablas de la base de datos.
5. Procesos y reglas de negocio implementados en el sistema.
6. Entrega, a través del sistema, de información relevante para el negocio (informes, reportes, gráficos).
7. Incorporar al menos una tecnología emergente (éstas pueden combinarse):
 - a. Análisis descriptivo (Ej, BI).
 - b. Data science (ej, machine learning).
 - c. Tecnologías de minería de datos (ej, web scrapping).
 - d. Componente de IA generativas.
 - e. Algunas imágenes en 3D.
 - f. Realidad aumentada o en su defecto virtualización
 - g. Arduino, raspberry, cámaras inteligentes, etc.
 - h. Machine learning MLOps: experiment tracking (MLflow), deployment de modelos (FastAPI, TorchServe).
 - i. Generative AI aplicada a procesos de negocio: chatbots internos fine-tuned, creación automática de documentación, synthetic data.
 - j. Edge/IoT + IA embebida: modelos TinyML en dispositivos Arduino/Raspberry.
 - k. Digital twins con motores 3D (Unity, Three.js) y realidad aumentada (WebXR).
 - l. Blockchain o DLT para trazabilidad si el caso lo amerita (supply chain, identidad).
 - m. Computación verde / eficiencia energética: medición de huella de carbono de la app.

Presentation y defensa

1. Storyboard y narrativa que conecte problema -> solución -> valor -> futuro.
2. Demostración en vivo con casos de uso reales.
3. Preguntas frecuentes anticipadas (costos, escalabilidad, riesgos, competencia).